Agentic AI-Powered Student Personal Assistant

Complete Technical Documentation & Teacher/Examiner Viva Handbook

A production-ready Multi-Agent Educational Assistant with Grounded RAG, Autonomous Topic Scheduling & Exam Intelligence.

Architecture: 10-Agent Autonomous Ecosystem

Backend: Python 3.11 / FastAPI / SQLAlchemy / Uvicorn

Frontend: React 19 / Vite / Tailwind CSS v4

Deployment: Docker Multi-Stage / Render Web Service

1. Executive Summary & Core Motivation

Traditional self-directed study frequently leads to study chaos: students do not know what topics to prioritize, how many days remain before exams, or which past year questions carry the highest marks. Furthermore, when students ask generic AI tools (such as public ChatGPT), models frequently **hallucinate** notation, definitions, and theorems that do not match the course professor's syllabus.

The **Agentic AI Student Assistant** solves these challenges by combining:

- **Grounded RAG (Retrieval-Augmented Generation):** Answers doubts strictly referencing the student's actual uploaded textbooks and slides with page/slide citations.
- **PYQ Exam Pattern Ranking:** Analyzes previous year question papers to surface "Must-Know" high-frequency questions.
- **Adaptive Dynamic Scheduling:** Generates personalized daily study sessions that automatically reschedule if a session is missed.
- **Multi-Factor Recommendation:** Continuously answers "*What should I study right now?*" using a weighted decision matrix.

2. System Architecture & Infrastructure Details

2.1 Component Architecture

```
[ Client Browser ] (React 19 + Vite + Tailwind CSS v4)
         │
         │  REST API Calls with JWT Bearer Token in Headers
         ▼
[ FastAPI Application Server ] (Python 3.11 / Uvicorn)
         ├── Authentication & Security (JWT, PBKDF2 Password Hashing)
         ├── 10-Agent Multi-Agent Ecosystem (Coordinated by Orchestrator)
         ├── Document Parsers (pypdf, python-docx, python-pptx)
         ├── Grounded RAG Engine (Semantic Keyword & Chunk Retrieval)
         └── Zero-Downtime Fallback Heuristics Engine
         │
         ├── [ Database Layer ] (PostgreSQL in Production / SQLite3 Local Fallback)
         └── [ Static SPA Mount ] (Serves React Production Dist Bundle)
```

2.2 Infrastructure & Deployment Pipeline

Layer	Technology	Key Technical Role
Frontend Runtime	React 19 + Vite	Virtual DOM reactivity, modular component structure, sub-second HMR.
Styling & Themes	Tailwind CSS v4	Modern design system with dual Light and Dark themes, persistent in `localStorage`.
API Gateway	FastAPI + Uvicorn	High-speed asynchronous request routing, Pydantic type validation, auto Swagger at `/docs`.
ORM & Storage	SQLAlchemy 2.0	Database abstraction layer with automatic fallback between PostgreSQL and SQLite.
Document Processors	pypdf, python-docx, python-pptx	Binary parsing of textbooks, notes, and lecture slide presentations.
Containerization	Docker Multi-Stage	Stage 1: `node:20-alpine` builds SPA; Stage 2: `python:3.11-slim` runs FastAPI server.
Cloud Platform	Render (render.yaml)	Continuous Git-triggered deployment with automated health check at `/health`.

3. The 10-Agent Multi-Agent Ecosystem

Rather than a single monolithic prompt, the platform decomposes student intelligence into 10 autonomous, collaborative micro-agents:

1. Agent Orchestrator `orchestrator.py`

Real-World Analogy: The Academic Dean & Central Dispatcher

Role: Manages the lifecycle and event flow between all other agents. When a student uploads notes or finishes a test, the Orchestrator sequences downstream agents and logs decisions to the Live AI Activity Stream.

2. Document Agent `document_agent.py`

Real-World Analogy: The University Head Librarian

Role: Extracts text from multi-format files (PDF, PPTX, DOCX, TXT), strips artifacts, and generates 500-character chunks with 100-character overlap for RAG indexing.

3. Syllabus Agent `syllabus_agent.py`

Real-World Analogy: The Curriculum Director

Role: Parses course outlines into structured Units/Modules and individual Topics, assigning estimated study hours and difficulty levels.

4. Exam Agent `exam_agent.py`

Real-World Analogy: The Deadline Strategist

Role: Computes days and hours remaining until exams and calculates the **Preparation Urgency Score**:

```
Urgency Score = min(100, max(10, (Target Score × 10) / max(1, Days Left)))
```

5. Adaptive Study Planner Agent `planner_agent.py`

Real-World Analogy: The Personal Academic Coach

Role: Formulates a day-by-day roadmap with Morning, Afternoon, and Evening slots. Dynamically shifts missed sessions forward to prevent exam cramming.

6. PYQ Analysis Agent `pyq_agent.py`

Real-World Analogy: The Exam Board Pattern Analyst

Role: Analyzes previous years' question papers, groups questions by unit, calculates topic recurrence rates, and marks high-yield questions.

7. Quiz Generation Agent `quiz_agent.py`

Real-World Analogy: The Examination Paper Setter

Role: Generates targeted MCQs with 4 options, distractors, correct answers, and detailed pedagogical explanations.

8. Evaluation Agent `evaluation_agent.py`

Real-World Analogy: The Diagnostic Examiner

Role: Evaluates student submissions, calculates mastery percentages, and flags any topic with <60% accuracy as a **Weak Area**.

9. Recommendation Agent `recommendation_agent.py`

Real-World Analogy: The Academic GPS Navigator

Role: Solves "*What should I study now?*" using a multi-factor priority formula:

```
Priority Weight = (0.40 × Exam Urgency) + (0.35 × PYQ Frequency) + (0.25 × Topic Weakness)
```

10. AI Tutor Agent `tutor_agent.py`

Real-World Analogy: The 24/7 Grounded Private Tutor

Role: Answers doubts using strict RAG semantic chunk retrieval, providing step-by-step explanations, citations (Document Name, Page/Slide number, snippet), and follow-up conceptual questions.

4. Grounded RAG (Retrieval-Augmented Generation) Workflow

The platform's Grounded RAG pipeline strictly separates external knowledge from verified student course material to eliminate AI hallucinations:

1. **Upload & Extraction:** The student uploads course files. The `DocumentAgent` parses the raw text from PDFs, DOCX, and PPTX presentations.
2. **Semantic Chunking:** Text is sliced into 500-character chunks with a 100-character overlapping sliding window. This preserves sentences and technical definitions across boundaries.
3. **Database Indexing:** Each chunk is stored with its `document_id`, `page_number`, and extracted content in the `document_chunks` table.
4. **Grounded Querying:** When a student submits a doubt in the AI Chat, the `TutorAgent` filters chunks by the active subject, ranks matches by keyword and semantic score, and constructs a grounded prompt.
5. **Citation Attribution:** The output includes the exact document name and page number, enabling students to verify answers directly in their textbooks.

5. Complete REST API Reference

Endpoint	Method	Purpose	Auth Required
`/api/v1/auth/register`	POST	Register new student	No
`/api/v1/auth/login`	POST	User login & JWT issuance	No
`/api/v1/auth/me`	GET	Get current student profile	Yes
`/api/v1/onboarding/stream`	POST	Initialize preset stream subjects	Yes
`/api/v1/subjects/`	GET/POST	Manage enrolled courses	Yes
`/api/v1/documents/upload`	POST	Upload PDF/PPTX/DOCX materials	Yes
`/api/v1/syllabus/{subject_id}`	GET	Get unit & topic breakdown	Yes
`/api/v1/exams/`	GET/POST	Track exams & countdowns	Yes
`/api/v1/study-plan/today`	GET	Get Morning/Afternoon/Evening tasks	Yes
`/api/v1/study-plan/reschedule`	POST	Dynamic reschedule of missed sessions	Yes
`/api/v1/pyq/analysis/{subject_id}`	GET	Recurring exam topic rankings	Yes
`/api/v1/quizzes/generate`	POST	Create adaptive practice quiz	Yes
`/api/v1/quizzes/{id}/submit`	POST	Evaluate test & diagnose weak areas	Yes
`/api/v1/analytics/`	GET	View streak, study hours & accuracy	Yes
`/api/v1/recommendations/next`	GET	Get "What to study now?" directive	Yes
`/api/v1/tutor/ask`	POST	Ask tutor doubt with RAG citations	Yes
`/api/v1/activity/stream`	GET	Live multi-agent activity stream	Yes
`/health`	GET	System health & DB status check	No

6. Teacher & Examiner Viva Voce: Top Questions & Answers

Q1. What is the core problem your project solves?

Self-directed study is often chaotic, exam deadlines are poorly managed, and students rely on generic AI that hallucinates. Our platform solves this using a 10-Agent Multi-Agent Ecosystem that provides grounded RAG doubt solving from actual textbooks, calculates PYQ exam weightage, and dynamically schedules daily study sessions.

Q2. Why did you use an Agentic Architecture instead of a single LLM prompt?

A single LLM prompt suffers from context pollution, high hallucination rates, and lack of state persistence. Our agentic architecture distributes tasks into modular micro-agents (e.g., ExamAgent handles countdowns, PYQAgent ranks recurring questions, TutorAgent answers doubts). This ensures mathematical accuracy and focused generative quality.

Q3. How does Grounded RAG work in your project, and what prevents hallucinations?

Uploaded course textbooks and lecture slides are parsed into 500-character chunks with 100-character overlap. The TutorAgent retrieves relevant chunks based on the student's query and strictly instructs the LLM to answer only using those snippets while providing exact citations (Document Name, Page #, Excerpt). If the textbook lacks the answer, the model explicitly states it.

Q4. How are PDF, PPTX, and DOCX files parsed?

We use dedicated Python binary parser libraries: `pypdf` for PDF streams, `python-pptx` for PowerPoint slide shapes and text frames, and `python-docx` for Word document paragraphs and tables. Extracted text is normalized and chunked in the database.

Q5. What is the formula used by the Exam Agent to compute Urgency?

`Urgency = min(100, max(10, (Target_Score × 10) / max(1, Days_Left)))`. As the exam approaches or if the target score is high (e.g., 95%), urgency scales toward 100%, prioritizing the subject across the dashboard.

Q6. How does the dynamic study planner reschedule missed study sessions?

When a student clicks "Reschedule", the StudyPlannerAgent finds past incomplete sessions, re-evaluates remaining days until the exam, sorts unmastered topics by PYQ weightage, and reallocates them into upcoming Morning, Afternoon, and Evening slots without exceeding daily study capacity.

Q7. How does database connectivity work in production vs local development?

We use an automatic dual-engine architecture in `backend/app/core/database.py`. On Render, it connects to PostgreSQL using `DATABASE_URL`. If that is unavailable or fails, it automatically falls back to an embedded SQLite3 database (`student_ai.db`), ensuring zero downtime.

Q8. What security mechanisms protect user passwords and endpoints?

Passwords are hashed using cryptographic PBKDF2-HMAC-SHA256 with 100,000 salt iterations. Authentication issues signed stateless JSON Web Tokens (JWT) containing user ID and expiration, validated on every request via FastAPI's `get_current_user` dependency.

Q9. How is the application deployed on Render using Docker?

We use a Multi-Stage Dockerfile: Stage 1 (`node:20-alpine`) builds the React SPA into static assets; Stage 2 (`python:3.11-slim`) installs backend requirements and starts FastAPI via Uvicorn. The backend mounts and serves the React assets on the single port assigned by Render.

Q10. What happens if external LLM APIs (Gemini/OpenAI) are unavailable?

The system includes a built-in Deterministic Fallback Engine. All 10 agents can execute autonomously using keyword heuristics, rule-based question parsing, and pre-compiled question banks. External API keys are optional.

Q11. How does the Evaluation Agent diagnose weak topics?

After quiz submission, the EvaluationAgent grades answers against the answer key. Topics with accuracy below 60% are flagged as "Weak Areas", which automatically boosts their priority in the Recommendation Agent.

Q12. What formula decides "What should I study right now?"

`Priority = (0.40 × Exam Urgency) + (0.35 × PYQ Topic Weight) + (0.25 × Diagnostic Weakness)` . The topic with the highest score is highlighted on the dashboard with a 1-click action button.